gemstone-rs Release Checklist

Crates, VSIX, GitHub release, and verification steps

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Release Checklist

- Before Release
- Publish
- Post-Release Verification
- Optional Live Smoke

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
```

- Refresh screenshots when the explorer or workbench UI changed:

```
make screenshots
```

- Run local verification:

```
make verify
make vscode-package
python3 docs/build_pdf_docs.py
```

Or use the dry-run release wrapper:

```
DRY_RUN=1 scripts/release_all.sh 0.2.0
```

`make verify` checks that PDF generation completes and produces non-empty PDF files. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms. The VSIX filename uses `vscode-gemstone-rs-workbench/package.json`; the crate release tag still uses the workflow `version` input.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
  -f version=0.2.0 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`
3. `gemstone-rs`
4. `gemstone-rs-cli`
5. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:

```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Manual end-to-end publishing remains available through the release wrapper:

```
PUBLISH_CRATES=1 PUBLISH_VSIX=1 CREATE_GITHUB_RELEASE=1 VERIFY_PUBLIC=1 DRY_RUN=0
scripts/release_all.sh 0.2.0
```

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.0
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX and `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.0
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, or run locally:

```
GS_RUN_LIVE_RUST=1 cargo test -p gemstone-rs live_ -- --test-threads=1
```

The live smoke coverage includes login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, and generated wrapper `printString`.

The `--test-threads=1` flag is intentional. GemStone GCI has process-global session state, so live tests should not be run concurrently.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.